Actividad 04

Tema: Algoritmos recursivos

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Docente	Tema	Duración
Harvey A. Norman Patrick	Algoritmos recursivos	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	22/10/2026	28/10/2026

Índice

1. Actividades **3**
1.1. Objetivo . . . 3
1.2. Definición . . . 3
1.3. Elementos de un algoritmo recursivo correcto . . . 3
1.4. El principio de inducción y la correctitud de la recursión . . . 3
1.5. Proceso de diseño de un algoritmo recursivo . . . 4
1.6. Tipos de recursión . . . 4
1.7. La pila de llamadas y el costo en memoria . . . 4
1.8. Recursión frente a iteración . . . 5
1.9. Relación con otras técnicas de diseño de algoritmos . . . 5
1.10. Ejemplos de diseño recursivo . . . 5
1.10.1. Ejemplo 1: Factorial (recursión lineal) . . . 5
1.10.2. Ejemplo 2: Sucesión de Fibonacci (recursión múltiple) . . . 5
1.10.3. Ejemplo 3: Torres de Hanoi (divide y vencerás recursivo) . . . 6
1.10.4. Ejemplo 4: Búsqueda binaria (recursión de cola) . . . 6
1.11. Resumen . . . 6

2. Referencias bibliográficas **6**

1. Actividades

1.1. Objetivo

Investigar el diseño de algoritmos recursivos: su definición, los elementos que debe tener todo algoritmo recursivo para ser correcto, los tipos de recursión, el proceso para diseñar uno a partir de un problema, su relación con otras técnicas de diseño (divide y vencerás, backtracking, programación dinámica) y sus ventajas y desventajas frente a una solución iterativa.

1.2. Definición

Un *algoritmo recursivo* es aquel que resuelve un problema invocándose a sí mismo sobre una o más instancias más pequeñas del mismo problema, hasta llegar a una instancia lo bastante simple como para resolverse directamente (Cormen, Leiserson, Rivest, y Stein, 2022). La recursión traslada al algoritmo la misma idea que una definición recursiva en matemáticas: un objeto se define en términos de una versión más simple de sí mismo (Wirth, 1976).

1.3. Elementos de un algoritmo recursivo correcto

Para que la recursión termine y produzca el resultado correcto, todo algoritmo recursivo necesita los siguientes elementos (Levitin, 2012):

- **Caso base (o caso trivial):** una o más instancias del problema que se resuelven directamente, sin nuevas llamadas recursivas. Es la condición de parada de la recursión.
- **Caso recursivo:** la regla que expresa la solución del problema en términos de la solución de una o más instancias más pequeñas del mismo problema.
- **Progreso hacia el caso base:** cada llamada recursiva debe operar sobre una instancia estrictamente más cercana al caso base (por ejemplo, un tamaño de entrada menor), de modo que el proceso termine en un número finito de pasos.
- **Combinación de resultados (cuando aplica):** cuando el problema se divide en más de una subinstancia, el algoritmo debe combinar las soluciones parciales para construir la solución del problema original.

Si falta el caso base, o si las llamadas recursivas no se acercan a él, el algoritmo entra en una recursión infinita que termina por agotar la memoria disponible para las llamadas (Brassard y Bratley, 1996).

1.4. El principio de inducción y la correctitud de la recursión

La correctitud de un algoritmo recursivo se demuestra de la misma forma que una demostración por *inducción matemática* (Cormen y cols., 2022):

- Se demuestra que el algoritmo es correcto para el caso base (caso base de la inducción).
- Se asume que el algoritmo es correcto para instancias más pequeñas que la actual (hipótesis inductiva).
- Se demuestra que, asumiendo lo anterior, el algoritmo también es correcto para la instancia actual (paso inductivo).

Esta correspondencia explica por qué la recursión resulta natural para problemas que ya tienen una definición inductiva, como el factorial, los números de Fibonacci o el recorrido de estructuras definidas recursivamente (listas, árboles).

1.5. Proceso de diseño de un algoritmo recursivo

Diseñar un algoritmo recursivo sigue, en general, los siguientes pasos (Levitin, 2012):

- **Paso 1:** identificar uno o más casos base, es decir, las instancias más pequeñas o simples del problema que se pueden resolver sin recursión.
- **Paso 2:** encontrar una forma de reducir el problema original a una o más instancias más pequeñas del mismo problema (la relación de recurrencia del algoritmo).
- **Paso 3:** expresar la solución del problema en función de la solución de esas instancias más pequeñas, incluyendo cómo combinar los resultados si hay más de una llamada recursiva.
- **Paso 4:** verificar que cada llamada recursiva progresa hacia el caso base (por ejemplo, que el tamaño de la entrada disminuye en cada llamada).
- **Paso 5:** analizar el algoritmo resultante planteando su relación de recurrencia de tiempo de ejecución y resolviéndola (por ejemplo, con el Teorema Maestro cuando aplica).

1.6. Tipos de recursión

- **Recursión directa:** un procedimiento se llama a sí mismo de forma explícita, como en el cálculo del factorial.
- **Recursión indirecta (o mutua):** un procedimiento A llama a un procedimiento B, y B llama de nuevo a A, formando un ciclo de llamadas entre dos o más procedimientos.
- **Recursión lineal:** cada llamada genera como máximo una nueva llamada recursiva (por ejemplo, el factorial o la búsqueda binaria). El árbol de llamadas resultante es, de hecho, una lista.
- **Recursión múltiple (o en árbol):** cada llamada genera más de una llamada recursiva, como en el cálculo ingenuo de Fibonacci o en el recorrido de un árbol binario. El árbol de llamadas crece en forma de árbol, no de lista.
- **Recursión anidada:** uno de los argumentos de la llamada recursiva es, a su vez, el resultado de otra llamada recursiva (por ejemplo, la función de Ackermann).
- **Recursión de cola (*tail recursion*):** la llamada recursiva es la última operación que realiza el procedimiento, sin trabajo pendiente después de que esta retorne. Muchos compiladores e intérpretes pueden optimizar este caso convirtiéndolo internamente en un bucle, evitando el crecimiento de la pila de llamadas (Brassard y Bratley, 1996).

1.7. La pila de llamadas y el costo en memoria

Cada llamada recursiva que todavía no ha retornado ocupa un marco de activación (*activation record*) en la pila de llamadas del programa, con sus parámetros, variables locales y la dirección de retorno (Cormen y cols., 2022). Esto tiene dos consecuencias prácticas:

- La **profundidad máxima de la recursión** (el número de llamadas pendientes en el peor caso) determina la complejidad espacial adicional que introduce la recursión, más allá del espacio que usaría una solución iterativa equivalente.
- Una recursión sin caso base, o cuyo progreso hacia el caso base es incorrecto, agota la pila de llamadas y produce un error de *stack overflow* (desbordamiento de pila).

1.8. Recursión frente a iteración

Todo algoritmo recursivo puede reescribirse como un algoritmo iterativo equivalente (y viceversa), ya que ambos modelos de cómputo tienen el mismo poder expresivo (Brassard y Bratley, 1996). La elección entre uno u otro es una decisión de diseño:

- **A favor de la recursión:** el código suele ser más corto, más cercano a la definición matemática o a la estructura de datos del problema (listas, árboles, grafos), y más fácil de demostrar correcto mediante inducción.
- **A favor de la iteración:** evita el costo en memoria y en tiempo de las llamadas de función (creación y destrucción de marcos de activación), y evita el riesgo de *stack overflow* en entradas grandes cuando el lenguaje o el entorno no garantiza optimización de recursión de cola.
- En la práctica, se prefiere la recursión cuando el problema tiene una estructura naturalmente recursiva y la profundidad de recursión es manejable, y la iteración cuando el rendimiento o el límite de pila son críticos.

1.9. Relación con otras técnicas de diseño de algoritmos

La recursión es el mecanismo que hace posible varias técnicas de diseño de algoritmos más generales (Cormen y cols., 2022):

- **Divide y vencerás:** divide el problema en subproblemas independientes del mismo tipo, los resuelve recursivamente y combina sus soluciones (por ejemplo, Merge Sort, Quicksort, la multiplicación de Karatsuba).
- **Backtracking (vuelta atrás):** explora recursivamente el espacio de soluciones de un problema, retrocediendo cuando una decisión parcial no puede llevar a una solución válida (por ejemplo, el problema de las N reinas o la resolución de laberintos).
- **Programación dinámica (enfoque top-down con memoización):** parte de la misma relación de recurrencia que una solución recursiva directa, pero almacena los resultados de las subinstancias ya resueltas para no recalcularlas, lo que puede reducir drásticamente el tiempo de ejecución de una recursión múltiple con subproblemas repetidos.

1.10. Ejemplos de diseño recursivo

1.10.1. Ejemplo 1: Factorial (recursión lineal)

Caso base: $0! = 1$.
Caso recursivo: $n! = n \cdot (n-1)!$ para $n \geq 1$.
Cada llamada reduce el problema en una unidad ($n \to n-1$), por lo que el progreso hacia el caso base es inmediato. Es un ejemplo de recursión lineal: el árbol de llamadas es, en realidad, una cadena de n llamadas.

1.10.2. Ejemplo 2: Sucesión de Fibonacci (recursión múltiple)

Caso base: $F(0) = 0$, $F(1) = 1$.
Caso recursivo: $F(n) = F(n-1) + F(n-2)$ para $n \geq 2$.
Cada llamada genera dos llamadas recursivas, por lo que el árbol de llamadas crece de forma exponencial y muchas subinstancias se recalculan varias veces (por ejemplo, $F(2)$ se calcula repetidamente al calcular $F(5)$). Este es el ejemplo clásico que motiva la programación dinámica con memoización como mejora directa sobre el diseño recursivo ingenuo.

1.10.3. Ejemplo 3: Torres de Hanoi (divide y vencerás recursivo)

Caso base: mover 0 discos no requiere ninguna acción.

Caso recursivo: para mover n discos del poste origen al poste destino usando un poste auxiliar: mover $n-1$ discos del origen al auxiliar, mover el disco restante del origen al destino, y mover los $n-1$ discos del auxiliar al destino.

Aquí la reducción no es solo en el tamaño del problema ($n \to n-1$), sino que además se necesitan dos llamadas recursivas para resolver una sola instancia, ilustrando cómo la recursión expresa de forma directa una estrategia de divide y vencerás.

1.10.4. Ejemplo 4: Búsqueda binaria (recursión de cola)

Caso base: el subarreglo está vacío (no se encontró el elemento), o el elemento central es igual al buscado.

Caso recursivo: si el elemento buscado es menor que el elemento central, buscar recursivamente en la mitad izquierda; si es mayor, buscar en la mitad derecha.

La llamada recursiva es la última operación del algoritmo y no queda trabajo pendiente después de que retorne, por lo que es un ejemplo de recursión de cola: puede convertirse directamente en un bucle `while` sin cambiar su comportamiento.

1.11. Resumen

Ejemplo	**Tipo de recursión**	**Relación de recurrencia**
Factorial	Lineal	$T(n) = T(n-1) + \Theta(1)$
Fibonacci (ingenuo)	Múltiple (en árbol)	$T(n) = T(n-1) + T(n-2) + \Theta(1)$
Torres de Hanoi	Múltiple (divide y vencerás)	$T(n) = 2T(n-1) + \Theta(1)$
Búsqueda binaria	De cola	$T(n) = T(n/2) + \Theta(1)$

El diseño recursivo, entonces, no es solo una forma distinta de escribir un algoritmo: define directamente la relación de recurrencia que describe su costo, la cual puede resolverse con los métodos revisados anteriormente (iteración, sustitución, árbol de recursión, ecuación característica o el Teorema Maestro) para conocer su complejidad.

2. Referencias bibliográficas

Akra, M., y Bazzi, L. (1998). On the solution of linear recurrence equations. *Computational Optimization and Applications*, *10*(2), 195–210.

Bentley, J. L., Haken, D., y Saxe, J. B. (1980). A general method for solving divide-and-conquer recurrences. *ACM SIGACT News*, *12*(3), 36–44.

Brassard, G., y Bratley, P. (1996). *Fundamentals of algorithmics*. Upper Saddle River, NJ: Prentice-Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Levitin, A. (2012). *Introduction to the design and analysis of algorithms* (3.ª ed.). Pearson.

Rosen, K. H. (2019). *Discrete mathematics and its applications* (8.ª ed.). McGraw-Hill Education.

Wirth, N. (1976). *Algorithms + data structures = programs*. Englewood Cliffs, NJ: Prentice-Hall.